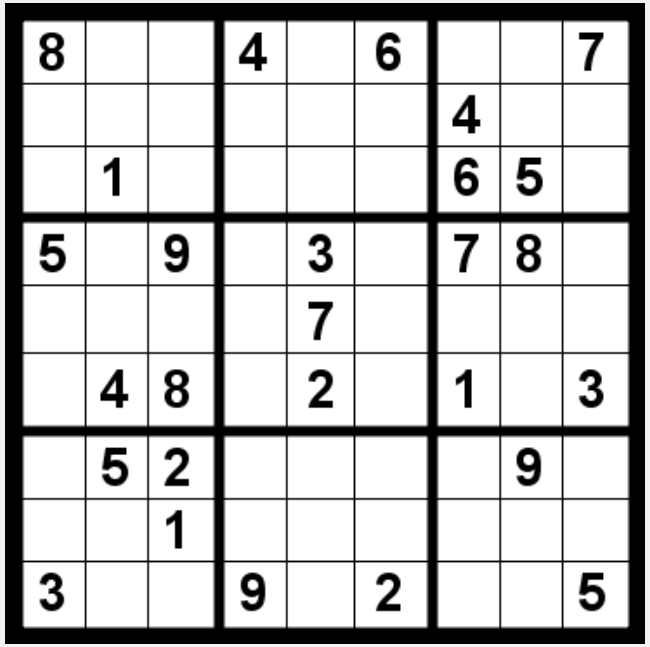
Constraint Processing:
Constraint Satisfaction/Optimization Problems

Constraint Processing by Rina Dechter

[https://www.sciencedirect.com/book/9781558608900/constraint-processing#book-info]

What is search for?

- **Search for *planning (sequence of actions)***
  - The path to the goal is the important thing
  - Paths have various costs, depths
- **Search for *assignment (identification)***
  - Assign values to variables while respecting certain constraints
  - The goal (complete, consistent assignment) is the important thing, not the path.



Constraint Processing

Constraint processing is a computational framework for solving problems defined by a set of variables, their possible values (domains), and constraints on allowable combinations of values.

- **Constraint Satisfaction**, where the goal is to find one or more assignments of values to variables that **satisfy all constraints**.
- **Constraint Optimization**, which is **not limited to satisfying all constraints**, but also aims to **optimize a given objective function** (e.g., cost, time, efficiency) **as much as possible** while still respecting the constraints.
- Applications include – Scheduling, Resource allocation, configuration problems, puzzle solving.

Constraint Satisfaction Problems

A constraint satisfaction problem consists of three components, X, D, and C:

X is a set of variables, $\{X_1, \ldots, X_n\}$.

D is a set of domains, $\{D_1, \ldots, D_n\}$, one for each variable.

C is a set of constraints that specify allowable combinations of values.

Each domain D_i consists of a set of allowable values, $\{v_1, \ldots, v_k\}$ for variable X_i.

Each state in a CSP is defined by an **assignment** of values to some or all of the variables

An assignment that does not violate any constraints is called a **consistent** or **legal** assignment.

A **complete assignment** is one in which every variable is assigned

A **partial assignment** is one that assigns values to only some of the variables.

A solution to a CSP is a consistent, complete assignment.

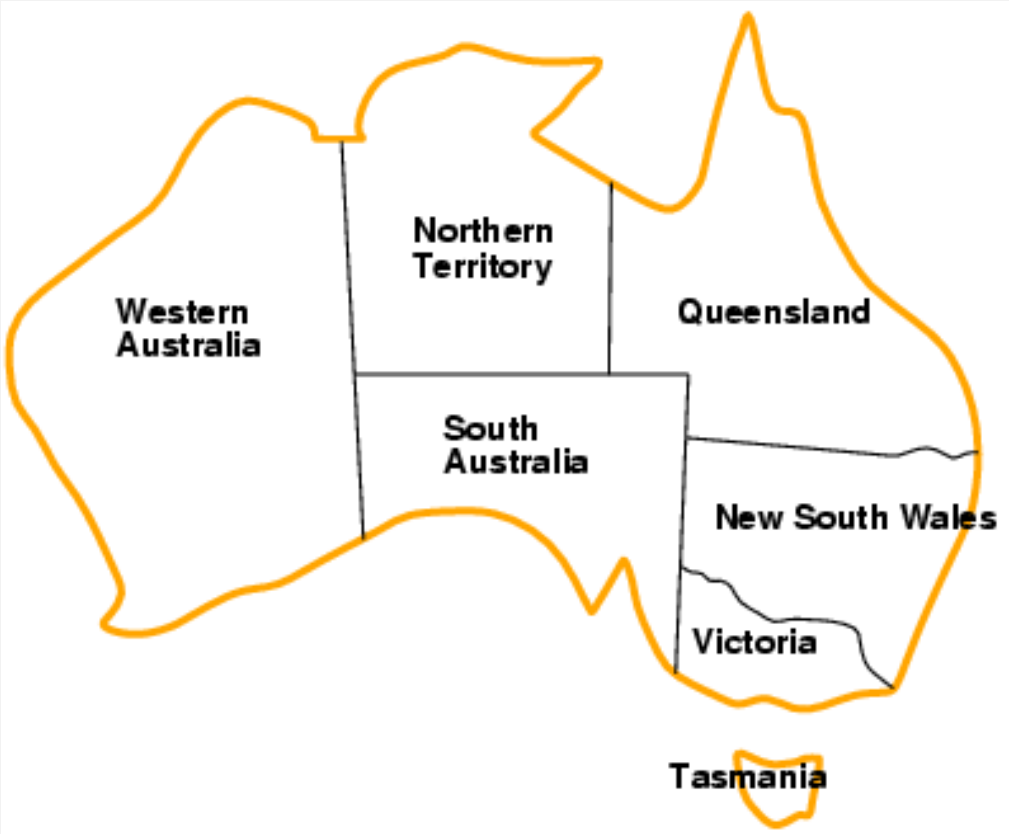
Constraint Satisfaction Problems

- *CSP formalism helps use general-purpose rather than problem-specific heuristics to enable the solution of complex problems.*
- CSPs yield a natural representation for a wide variety of problems
- CSP solvers can be faster than state-space searchers because the CSP solver can **quickly eliminate large portion of the search space**.

- Applications:
  - Scheduling problems
    - Job shop scheduling
    - Scheduling the Hubble Space Telescope
  - Floor planning for VLSI
  - Map coloring

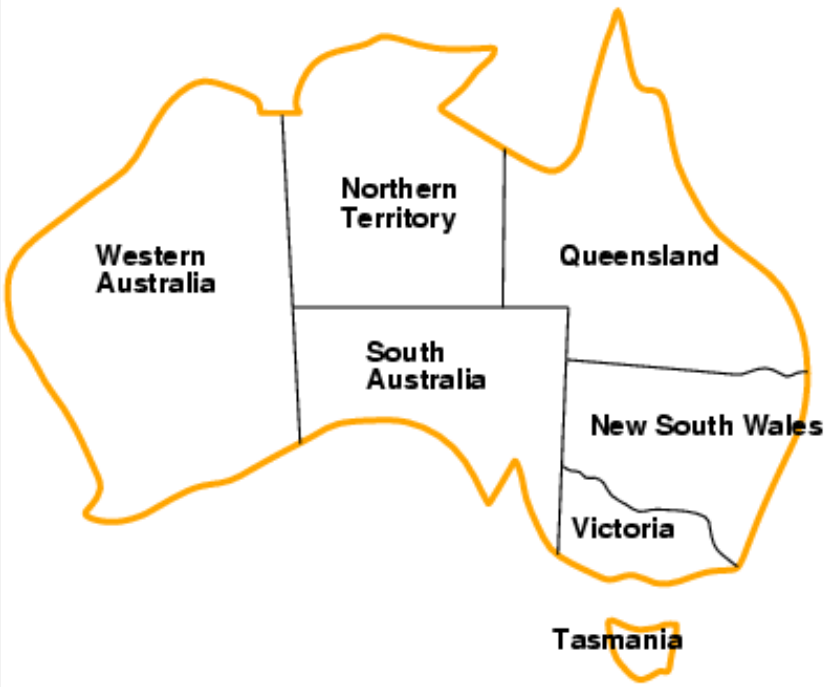
Example: Map Coloring

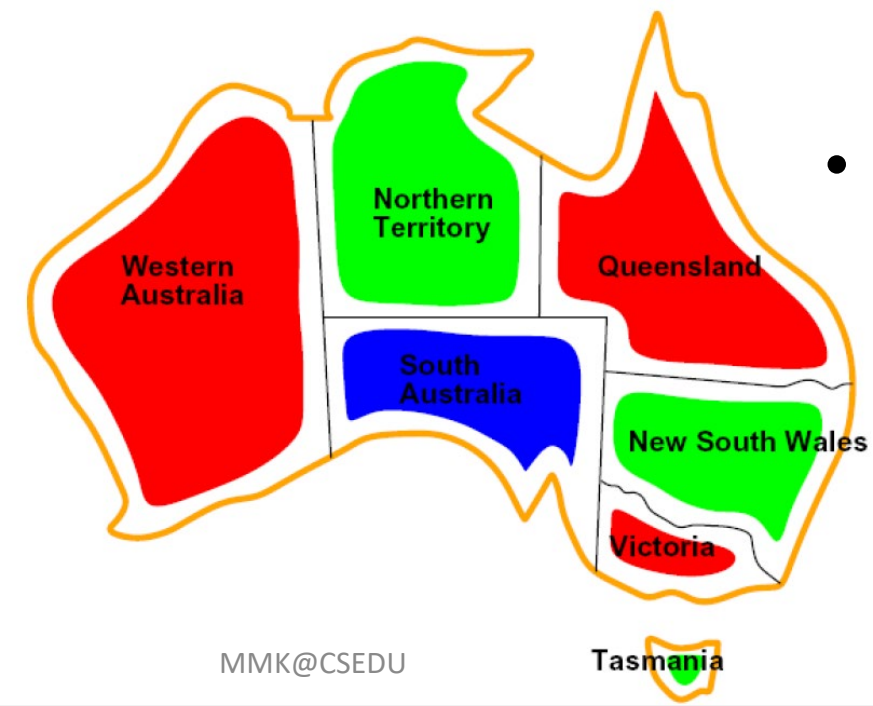
- **Variables:** WA, NT, Q, NSW, V, SA, T
- **Domains:** {red, green, blue}
- **Constraints:** adjacent regions must have different colors
  e.g., WA ≠ NT, or (WA, NT) in {(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)}



Example: Map Coloring

- Let SA = blue
- Without taking advantage of **constraint propagation**, a search procedure would have to consider $3^5 = 243$ assignments for the five neighboring variables
- with constraint propagation we never have to consider blue as a value, we have only $2^5 = 32$ assignments to look at, a reduction of 87%





- **Solutions** are *complete* and *consistent* assignments

Real-World CSPs

- Assignment problems
  - e.g., who teaches what class
  - hard constraint - no prof can teach 2 classes at the same time.
  - preference constraints – Jan prefers teaching in the morning while Alex prefers teaching in the afternoon (soft constraints/costs). For example, an afternoon slot for Jan can cost 2 points whereas a morning slot costs 1.

Real-World CSPs

- Timetable problems
  - e.g., which class is offered when and where?
  - Classroom size & enrollment can put hard constraints on where a class can be held
  - Avoiding simultaneous scheduling of two related classes can be a soft constraint

Real-World CSPs

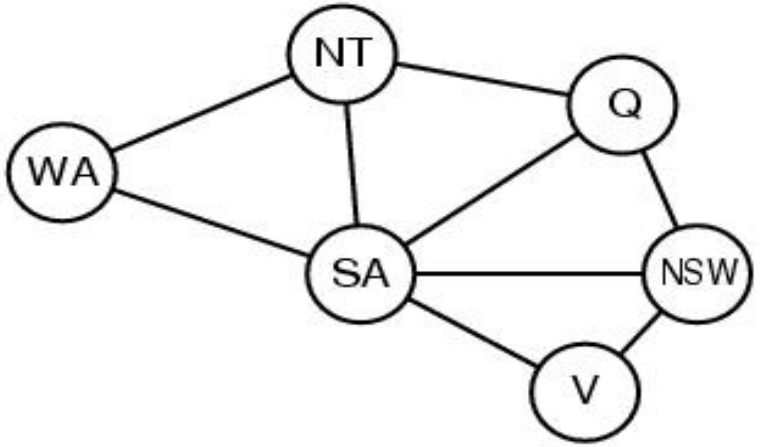
- Transportation Scheduling
  - E.g. airline scheduling Flights F_1,F_2 with capacities 165 and 385
  - Domains D_1=[0,165] D_2=[0,385]
  - Suppose we have an additional constraint that the two flights must carry 420 people: ie F_1+F_2=420
  - Then we can reduce the domains to:

    D_1=[35,165] D_2=[255,385]

    Enforcing Bounds consistency

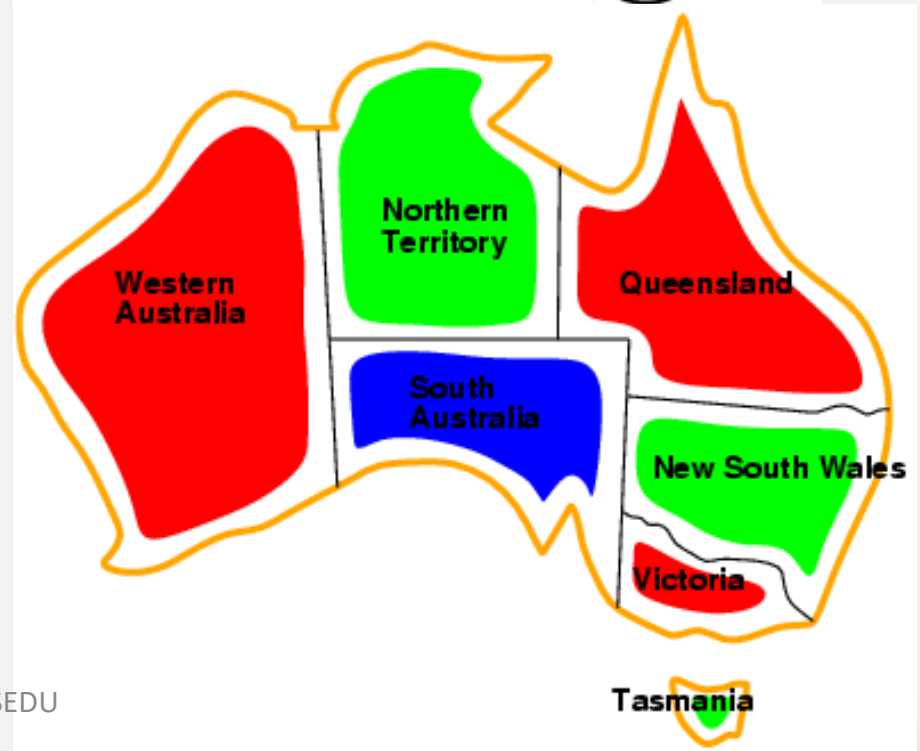
Constraint graph

- *Binary CSP:* each constraint relates two variables
- *Constraint graph:*
  - *nodes* are variables
  - *arcs* are constraints



- CSP benefits
  - Standard representation pattern: variables with values
  - Generic goal and successor functions
  - Generic heuristics (no domain specific expertise).

- Graph can be used to simplify search.
  - e.g. Tasmania is an independent subproblem.



Varieties of CSP Formalism

- Discrete Domain
  - finite domains:
    - *n* variables, domain size *d* → $O(d^n)$ complete assignments
    - The 8-queens problem where the variables Q_1, ..., Q_8 are the positions of each queen in columns 1,..., 8
    - Each variable has the domain D_i = {1, 2, 3, 4, 5, 6, 7, 8}.

Varieties of CSP Formalism

- Discrete Domain
  - infinite domains:
    - Set of integers, strings, etc.
    - Job-scheduling problem without a deadline, then there would be infinite number of start times for each variable
    - With infinite domains, it is no longer possible to describe constraints by enumerating all allowed combinations of values.

    - Instead, a constraint language must be used that understands constraints such as $T_1 + d_1 \leq T_2$ directly, without enumerating the set of pairs of allowable values for (T_1, T_2).

Varieties of CSP Formalism

Continuous Domain

- Continuous-domain CSPs are integral to **operations research**, notably for real-world applications like the **precise scheduling of the Hubble Space Telescope**,
  - These continuous variables must conform to astronomical, precedence, and power constraints.
- **Linear programming** is a prominent example of continuous-domain CSPs, characterized by linear equalities or inequalities and solvability in polynomial time based on the number of variables.
- Research in CSPs also includes problems with alternative constraints and goals, such as quadratic programming and second-order conic programming, demonstrating the field's expansive scope.

Varieties of constraints

- *Unary* constraints involve a single variable,
  - e.g., SA ≠ green
- *Binary* constraints involve pairs of variables,
  - e.g., SA ≠ WA
- *Higher-order* constraints involve 3 or more variables
  - e.g., crypt-arithmetic column constraints
- *Preference* (soft constraints) e.g. *red* is better than *green* can be represented by a cost for each variable assignment
  - Constrained optimization problems.

Varieties of constraints

Global Constraints

- A constraint involving an arbitrary number of variables is called a **global constraint**.
- The name is traditional but confusing because it need not involve *all* the variables in a problem – *Alldiff* in Sudoku problems

8			4		6			7
						4		
	1					6	5	
5		9		3		7	8	
				7				
	4	8		2		1		3
	5	2					9	
		1						
3			9		2			5

Constraint propagation: Inference in CSP

- Constraint propagation
    - may be intertwined with search,
    - or it may be done as a **preprocessing** step, before search starts

- Sometimes this preprocessing can solve the whole problem, so no search is required at all
    - by reducing every domain to size 1
- sometimes finding that the CSP cannot be solved
    - by reducing some domain to size 0

Enforcing Consistency

- Local Consistency

- the process of enforcing **local consistency** in each part of the graph causes inconsistent values to be eliminated throughout the graph.

  - Node consistency
  - Arc consistency
  - Path consistency
  - K-consistency

Enforcing Consistency – Node Consistency

- Node consistency refers to a form of local consistency applied to individual variables and their unary constraints.

A variable in a CSP is **node-consistent** if:

- Every value in its domain satisfies all unary constraints on that variable.
- That is, for variable X with domain D_X, and a unary constraint C(X).
- every value $v \in D_X$ must satisfy C(X).

Example

- Variable A has domain $D_A = \{1,2,3,4\}$
- Unary constraint: $A > 2$

To make A node-consistent:

- Remove any values from D_A that do not satisfy the constraint.
- New domain: $D_A=\{3,4\}$

Enforcing Consistency – Arc Consistency

A variable X is **arc-consistent** with another variable Y **(denoted as arc $X \rightarrow Y$)** if:

For every value x in the domain of X, **there exists at least one value y** in the domain of Y such that the **binary constraint between X and Y is satisfied**.

If this condition is not met, the value x should be removed from X's domain.

- Variables:
  $X \in \{1, 2, 3\}$,
  $Y \in \{2, 3\}$
- Constraint: $X < Y$

Check arc $X \rightarrow Y$:

- $X = 1 \rightarrow Y = 2$ or 3: ✅ (kept)
- $X = 2 \rightarrow Y = 3$: ✅ (kept)
- $X = 3 \rightarrow$ no Y satisfies $3 < Y$: ❌ (remove 3)

New domain of X: $\{1, 2\}$

Enforcing Consistency – Arc Consistency

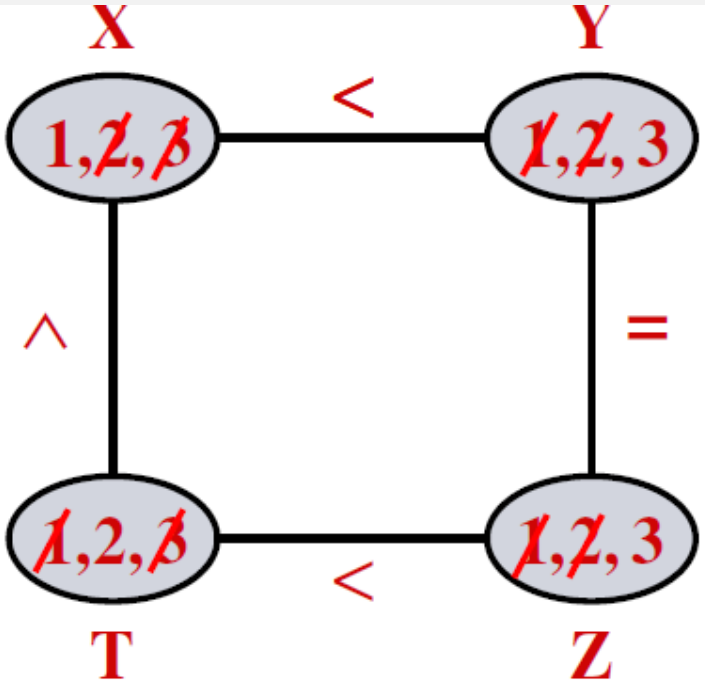
$1 \le X, Y, Z, T \le 3$

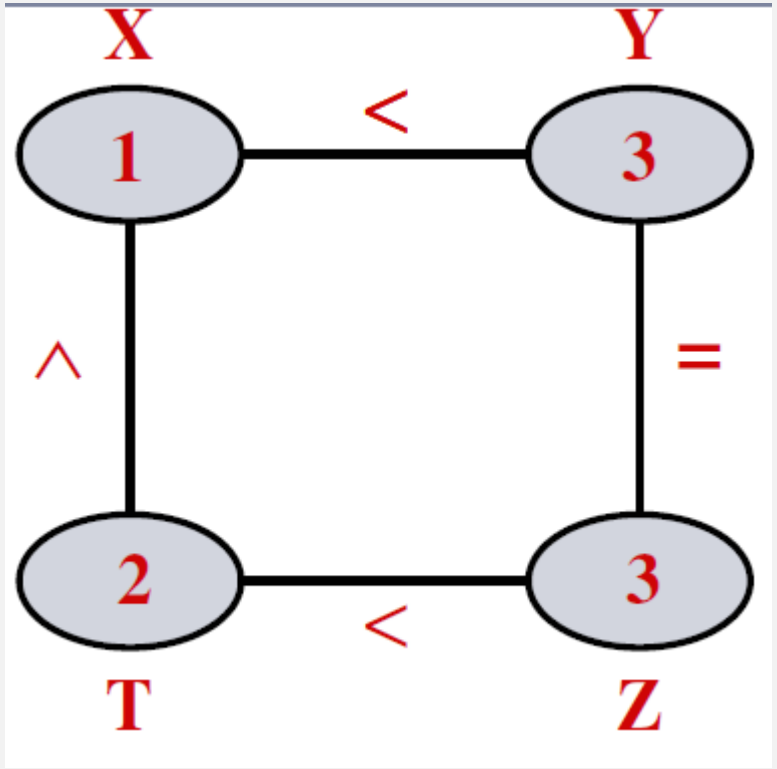
$X < Y$

$Y = Z$

$T < Z$

$X \le T$





Arc Consistency (AC – 3 Algorithm)

```
function AC-3( csp) returns false if an inconsistency is found and true otherwise
  inputs: csp, a binary CSP with components (X, D, C)
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REVISE(csp, X_i, X_j) then
      if size of D_i = 0 then return false
      for each X_k in X_i.NEIGHBORS - {X_j} do
        add (X_k, X_i) to queue
  return true
```

$$queue \leftarrow queue \cup \{(x_i, x_j), (x_j, x_i)\}$$

```
function REVISE( csp, X_i, X_j) returns true iff we revise the domain of X_i
  revised ← false
  for each x in D_i do
    if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j then
      delete x from D_i
      revised ← true
  return revised
```

Figure 6.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name "AC-3" was used by the algorithm's inventor (Mackworth, 1977) because it's the third version developed in the paper.

Arc Consistency (AC – 3 Algorithm)

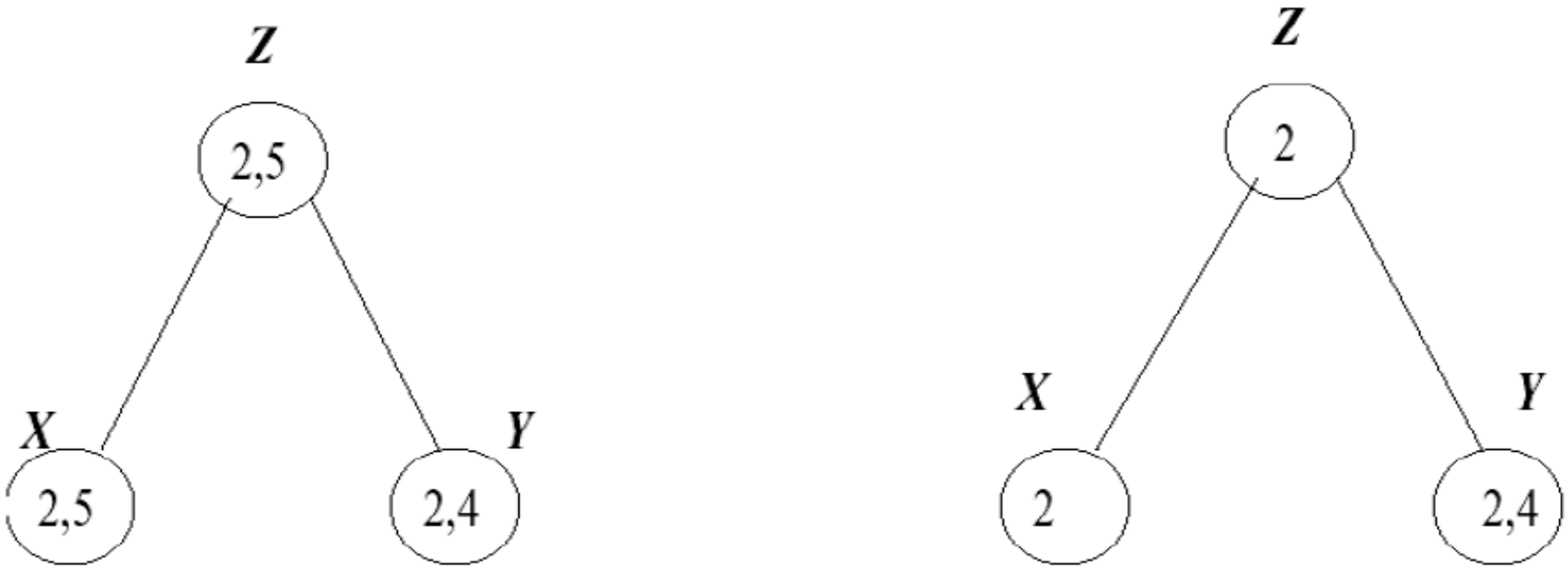
AC-3($\mathcal{R}$)

input: a network of constraints $\mathcal{R} = (X, D, C)$
output: $\mathcal{R}'$ which is the largest arc-consistent network equivalent to $\mathcal{R}$

1. **for** every pair $\{x_i, x_j\}$ that participates in a constraint $R_{ij} \in \mathcal{R}$
2. $\quad$ $queue \leftarrow queue \cup \{(x_i, x_j), (x_j, x_i)\}$
3. **endfor**
4. **while** $queue \neq \{\}$
5. $\quad$ select and delete (x_i, x_j) from $queue$
6. $\quad$ $Revise((x_i), x_j)$
7. $\quad$ **if** $Revise((x_i), x_j)$ causes a change in D_i
8. $\quad\quad$ **then** $queue \leftarrow queue \cup \{(x_k, x_i), i \neq k\}$
9. $\quad$ **endif**
10. **endwhile**

Arc Consistency (AC – 3 Algorithm)

Example: A three variable network, with two constraints: z divides x and z divides y (a) before and (b) after AC-3 is applied.



Enforcing Consistency – Arc Consistency

- AC-3 provides a new CSP that is

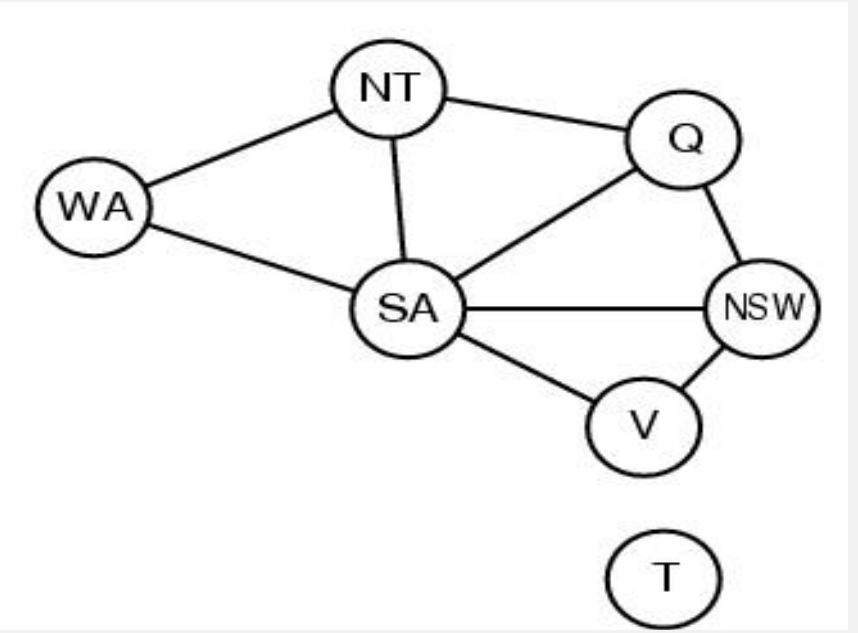
- **equivalent to the original CSP**—they both have the same solutions
- arc-consistent CSP will in most cases be faster to search because its variables have smaller domains.

- Home Task: Explore 4 algorithms (i.e. AC-1, AC-2, AC-3 and AC-4) that have been used to enforce arc consistency on a constraint graph representing a CSP, and prepare distinguishable exemplary scenario for each of them.

Enforcing Consistency – Arc Consistency

- Generalized Arc Consistency/ Hyperarc Consistency
- extend the notion of arc consistency to handle ***n-ary*** rather than just ***binary*** constraints
- For example, if all variables have the domain {0, 1, 2, 3},
- then to make the variable X consistent with the constraint X < Y < Z -
  - need to eliminate 2 and 3 from the domain of X
  - Otherwise the constraint cannot be satisfied when X is 2 or 3.

Enforcing Consistency – Path Consistency



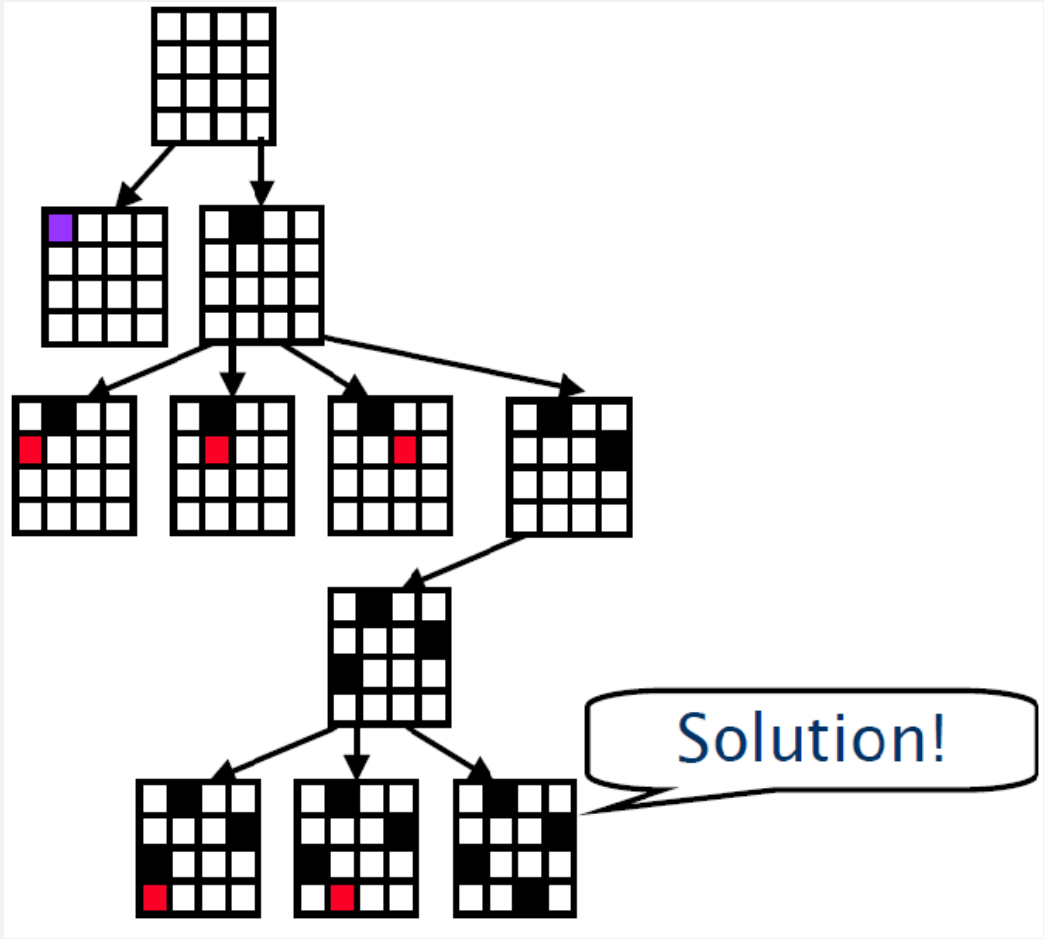
- Arc consistency can do nothing because every

  Variable is already arc consistent; when Domain {R, B}

- A two-variable set $\{X_i, X_j\}$ is path-consistent with respect to a third variable X_m if-

- for every assignment $\{X_i = a, X_j = b\}$ consistent with the constraints on $\{X_i, X_j\}$, there is an assignment to X_m that satisfies the constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$.
- This is called path consistency because one can think of it as looking at a path from X_i to X_j with X_m in the middle.

Backtracking search for solving CSP



Backtracking search for solving CSP:

Improving Backtracking

- Before search: (reducing the search space)
  - Arc-consistency, path-consistency, k-consistency
- During search:

  Look Ahead schemes
  - Value Ordering
  - Variable ordering (if not fixed)

  Look-back schemes
  - Buckjump
  - Dependency-directed backtacking

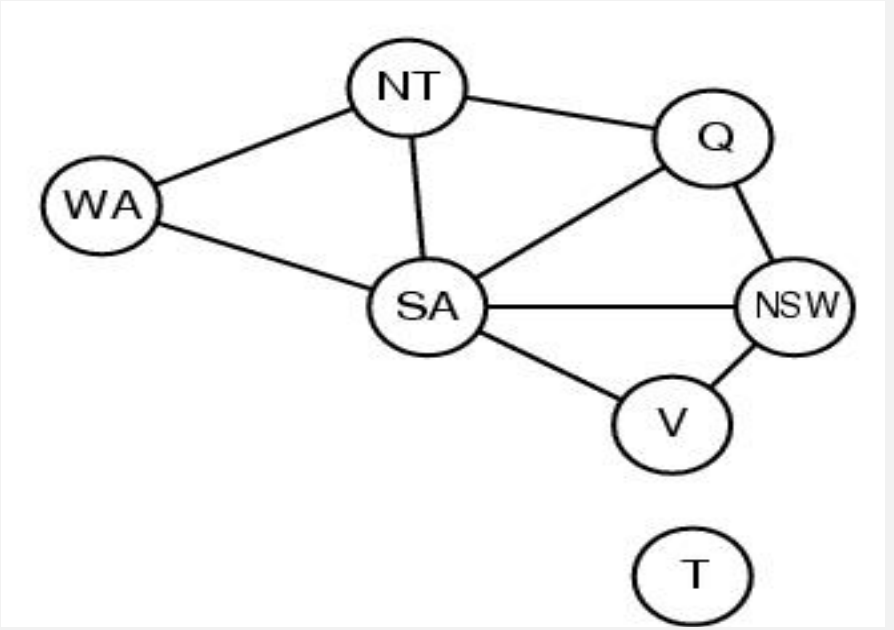
Backtracking search for solving CSP

- Which variable should be assigned next (SELECT-UNASSIGNED-VARIABLE), and in what order should its values be tried (ORDER-DOMAIN-VALUES)?

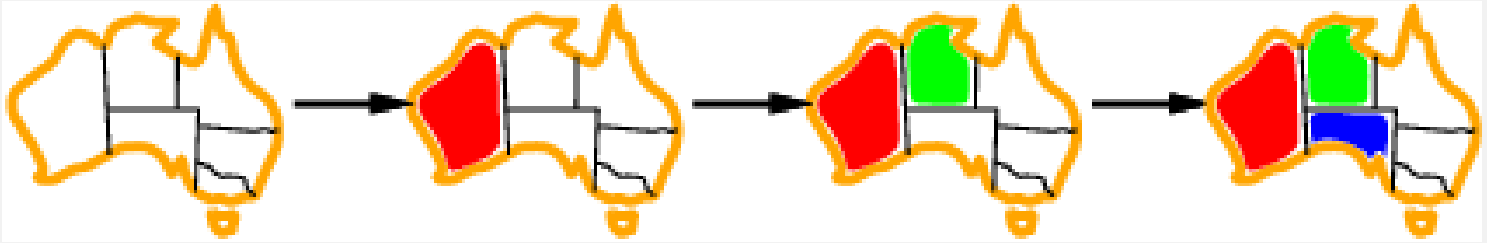
- What inferences should be performed at each step in the search (INFERENCE)?

- When the search arrives at an assignment that violates a constraint, can the search avoid repeating this failure?

Heuristic 1: Most constrained variable



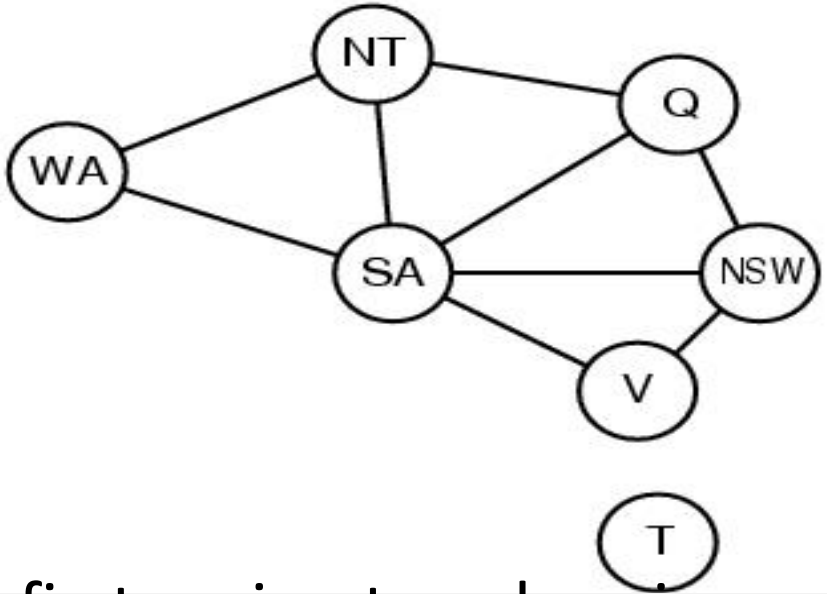
- Most constrained variable:
  choose the variable with the fewest legal values



- a.k.a. *minimum remaining values (MRV) or Fail-Fast* heuristic

- If some variable X has no legal values left, the MRV heuristic will select X and failure will be detected immediately—avoiding pointless searches through other variables.
- The MRV heuristic usually performs better than a random or static ordering, sometimes by a factor of 1,000 or more, although the results vary widely depending on the problem.

Heuristic 2:Degree Heuristic

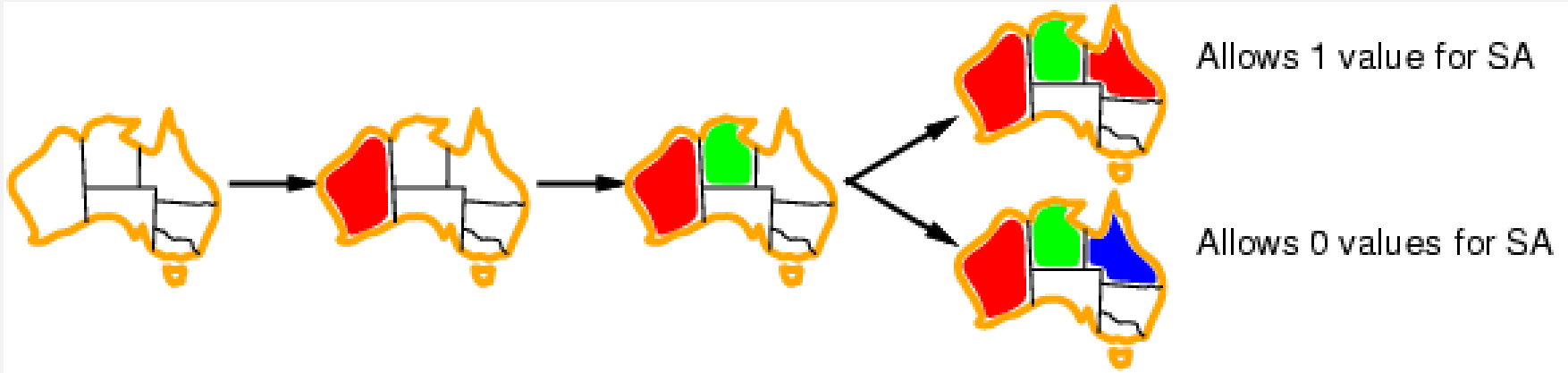


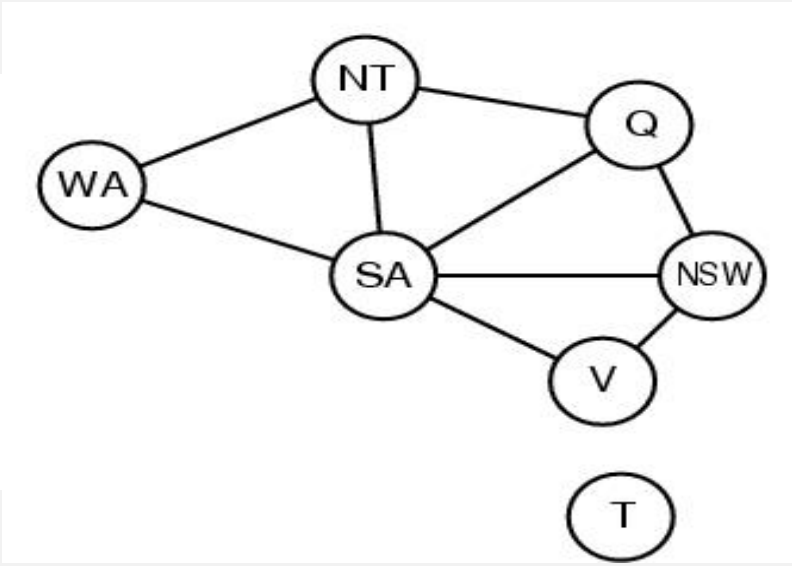
- The MRV heuristic doesn’t help at all in choosing the first region to colour in Australia, because initially every region has three legal colours.

- It attempts to reduce the branching factor on future choices by selecting the variable that is involved in the largest number of constraints on other unassigned variables.
- The degree heuristic can be useful as a tie-breaker.

Heuristic 3: Least constraining value

- Given a variable, choose the least constraining value:
  - the one that rules out the fewest values in the remaining variables





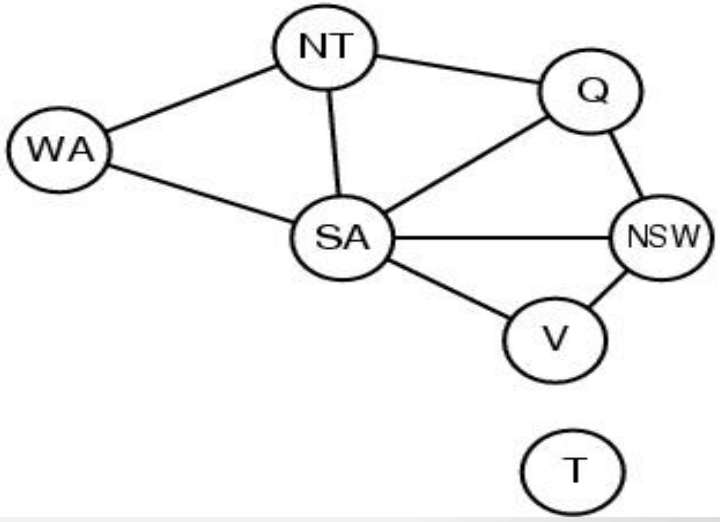
- the heuristic is trying to leave the maximum flexibility for subsequent variable assignments.

Combining these heuristics makes 1000 queens feasible

Interleaving search and inference – Forward Checking

- **Initiation**: Activated upon the assignment of a value to a variable X in a CSP.
- **Objective**: Ensures arc consistency for each unassigned variable *Y* connected to *X*, post *X*'s assignment.
- **Process**: Removes from *Y*'s domain any value incompatible with *X*'s assigned value.
- **Scope**: Operates locally, affecting only unassigned variables directly constrained by *X*.
- **Outcome**: Reduces the search space by pruning inconsistent values early, facilitating easier problem-solving.
- **Preprocessing Redundancy**: If arc consistency is pre-established, forward checking's impact is minimized or nullified.
- **Operational Efficiency**: Balances implementation simplicity with significant search space reduction benefits.
- **Practicality**: Offers a pragmatic approach to dynamically maintaining arc consistency in a CSP's evolving state.

	WA	*NT*	*Q*	*NSW*	*V*	*SA*	*T*
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After *WA=red*	(R)	G B	R G B	R G B	R G B	G B	R G B
After *Q=green*	(R)	B	(G)	R B	R G B	B	R G B
After *V=blue*	(R)	B	(G)	R	(B)		R G B

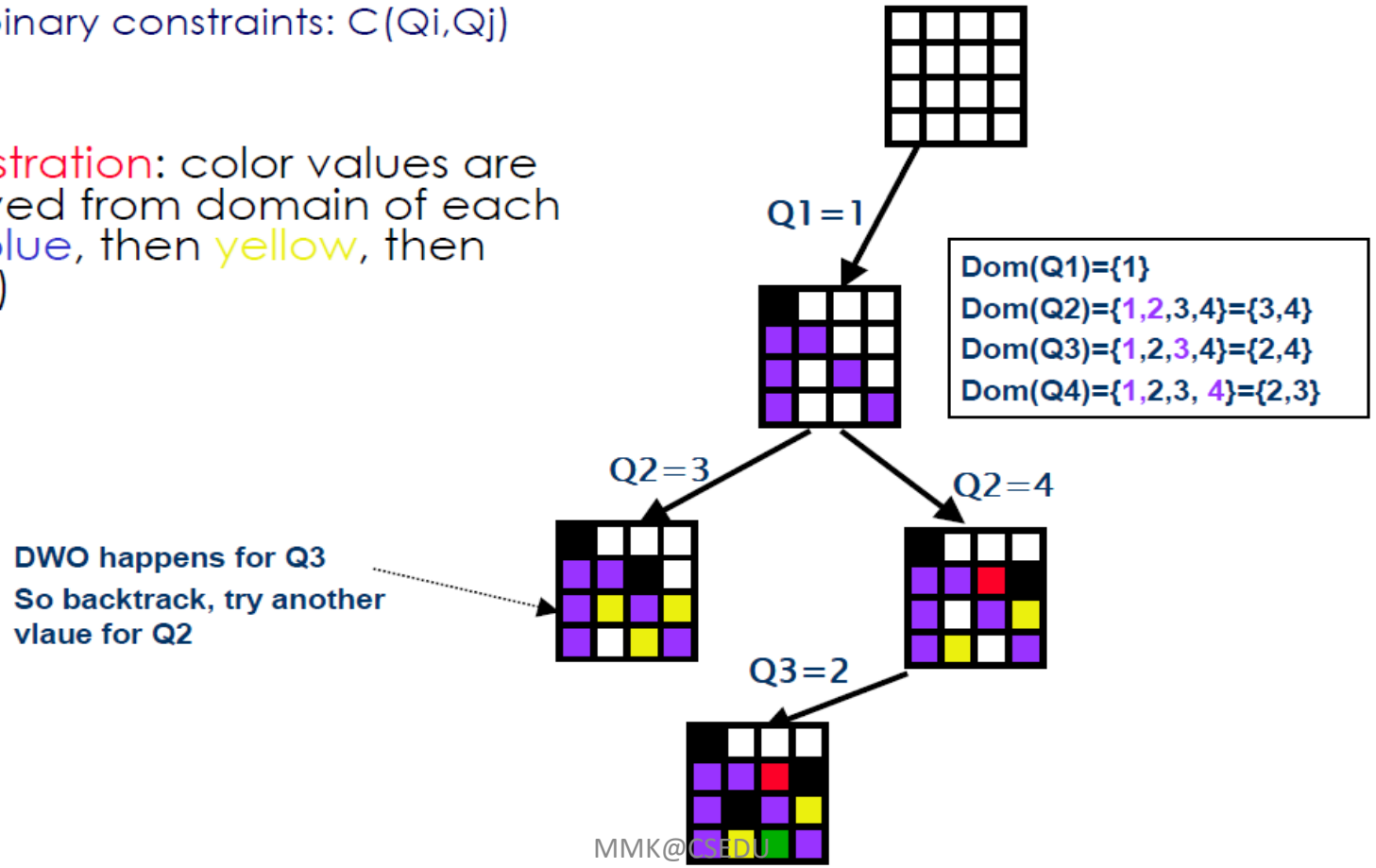


Interleaving search and inference – Forward Checking

FC Example.

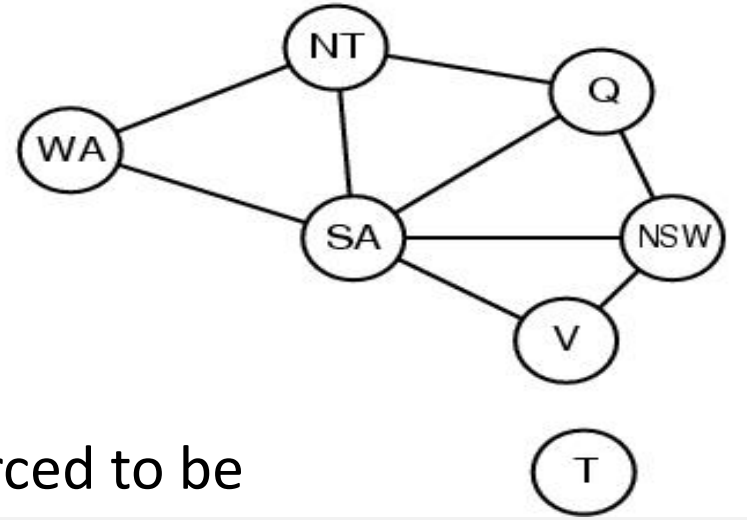
- 4X4 Queens
  - Q1,Q2,Q3,Q4 with domain {1..4}
  - All binary constraints: C(Qi,Qj)

- FC illustration: color values are removed from domain of each row (blue, then yellow, then green)



Forward Checking issues to Maintaining Arc Consistency (MAC)

	WA	*NT*	*Q*	*NSW*	*V*	*SA*	*T*
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After *WA=red*	(R)	G B	R G B	R G B	R G B	G B	R G B
After *Q=green*	(R)	B	(G)	R B	R G B	B	R G B
After *V=blue*	(R)	B	(G)	R	(B)		R G B



- In row 3, when WA is red and Q is green, both NT and SA are forced to be blue.
- Forward checking does not look far enough ahead to notice that this is an inconsistency: NT and SA are adjacent and so cannot have the same value.

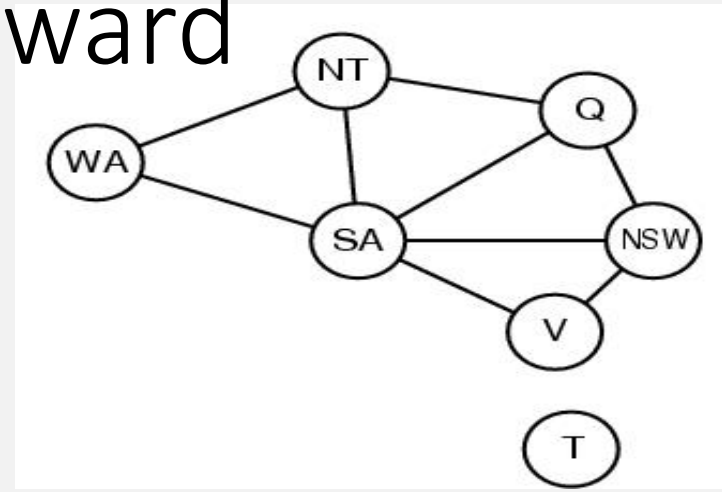
Interleaving search and inference - Maintaining Arc Consistency (MAC)

- After a variable X_i is assigned a value, the INFERENCE procedure calls AC-3, but instead of a queue of all arcs in the CSP, we start with only the arcs (X_j, X_i) for all X_j that are unassigned variables that are neighbors of X_i.

- From there, AC-3 does constraint propagation in the usual way, and if any variable has its domain reduced to the empty set, the call to AC-3 fails and we know to backtrack immediately.

- MAC is strictly more powerful than forward checking because forward checking does the same thing as MAC
- on the initial arcs in MAC's queue; but unlike MAC, forward checking does not recursively
- propagate constraints when changes are made to the domains of variables.

Intelligent backtracking: Looking backward



Q = red, NSW = green, V = blue, T = red

For SA every value violates a constraint,

so we back up to T and try a new color for T ???

A more intelligent approach to backtracking is to backtrack to a variable that might fix the problem—a variable that was responsible for making one of the possible values of SA impossible.

Conflict Set for SA: Q = red, NSW = green, V = blue

The **backjumping** method backtracks to the most recent assignment in the conflict set: instead of T, it will work on V

Check- how forward checking can produce the conflict set without any extra work!!

Also, have a look at the impact of backjumping and MAC

Local Search for CSPs : basics of local search

- The search algorithms that we have seen so far are designed to explore search spaces systematically.
- This systematicity is achieved by keeping one or more paths in memory and by recording which alternatives have been explored at each point along the path.
- When a goal is found, the path to that goal also constitutes a solution to the problem.
- the path to the goal is irrelevant in many problems (e.g. CSPs, job shop etc), thus the paths are not retained.
- Operate using a single current node (rather than multiple paths) and generally move only to neighbours of that node.
- they use very little memory—usually a constant amount.
- they can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

Local Search for CSPs : Min-conflict Heuristic

- In choosing a new value for a variable, the most obvious heuristic is to select the value that results in the minimum number of conflicts with other variables.

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
          max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for csp
  for i = 1 to max_steps do
      if current is a solution for csp then return current
      var ← a randomly chosen conflicted variable from csp.VARIABLES
      value ← the value v for var that minimizes CONFLICTS(var, v, current, csp)
      set var = value in current
  return failure
```

Figure 6.8 The MIN-CONFLICTS algorithm for solving CSPs by local search. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

Local Search for CSPs : Min-conflict Heuristic

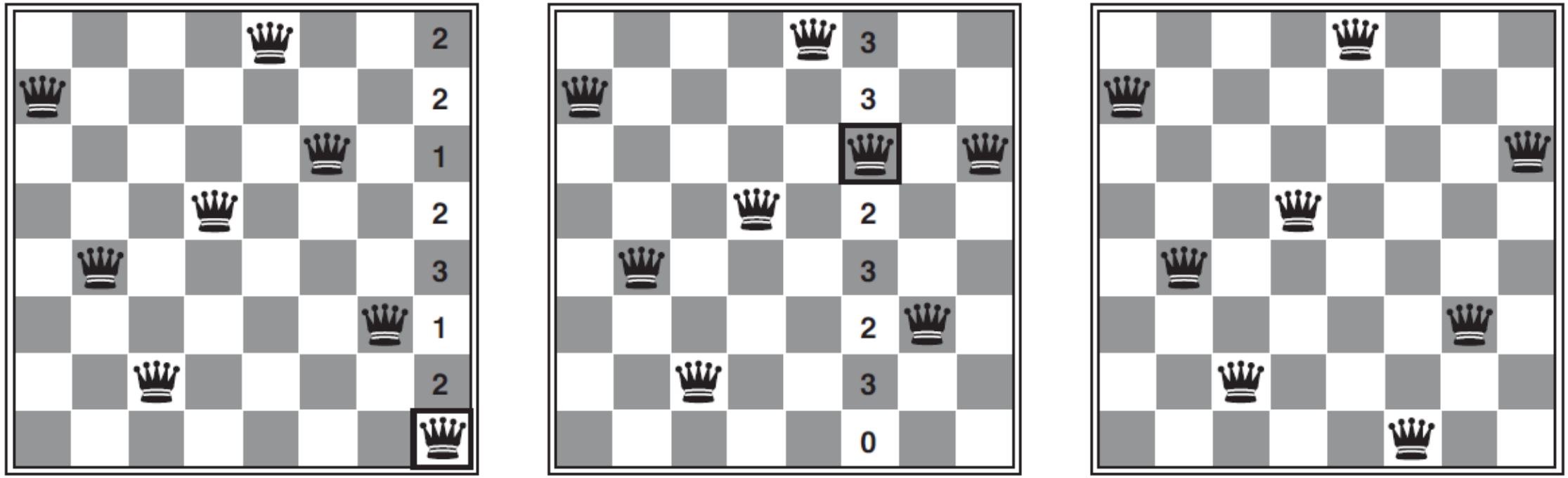


Figure 6.9 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Local Search for CSPs : Min-conflict Heuristic

- Min-conflicts is surprisingly effective for many CSPs.
- Amazingly, on the n-queens problem, if you don't count the initial placement of queens, the run time of min-conflicts is roughly *independent of problem size*.
- It solves even the *million*-queens problem in an average of 50 steps (after the initial assignment).

- This remarkable observation was the stimulus leading to a great deal of research in the 1990s on local search.
- it has been used to schedule observations for the Hubble Space Telescope, reducing the time taken to schedule a week of observations ***from three weeks (!) to around 10 minutes***.